IBM ICE Day Ideathon

Roadkill Hotspots & Policy Advocacy

Innovation For A Cause





Lokesh | Harshit | Meheak

Saving Animal lives through AI-Driven Solutions



AGENDA

Roadkill

Protecting Animals: Innovation for a Cause

Vision

Technical Overview

Coding & Demonstration

How will it work?

Implementation Strategy

Advanced Features

Business & Sustainability Plan

Expected Impact





Roadkill?

When a **being** *other than the sapiens*, is **ran over** by a locomotive machine driven by a sapien is basically a **roadkill**.

**The following images contain graphic content of deceased animals and may be distressing to some viewers.




alamy
alamy







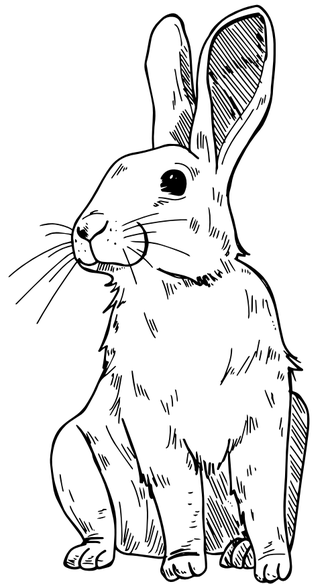






Protecting Animals

- Frequent roadkill incidents due to **speeding vehicles**, 340+ in 10months at MTR, Niligiri [30% reptiles, 28% amphibians, 24% birds & 18% mammals.
- **No real-time alert systems** for animal presence on roads.
- Lack of preventive measures results in **unnoticed deaths**.





Our Vision



- Prevent animal deaths using **ML** & **Computer Vision**.
- Advocate for AI-based **road safety policies**.
- Collaborate with **automobile industries** & **governments**.



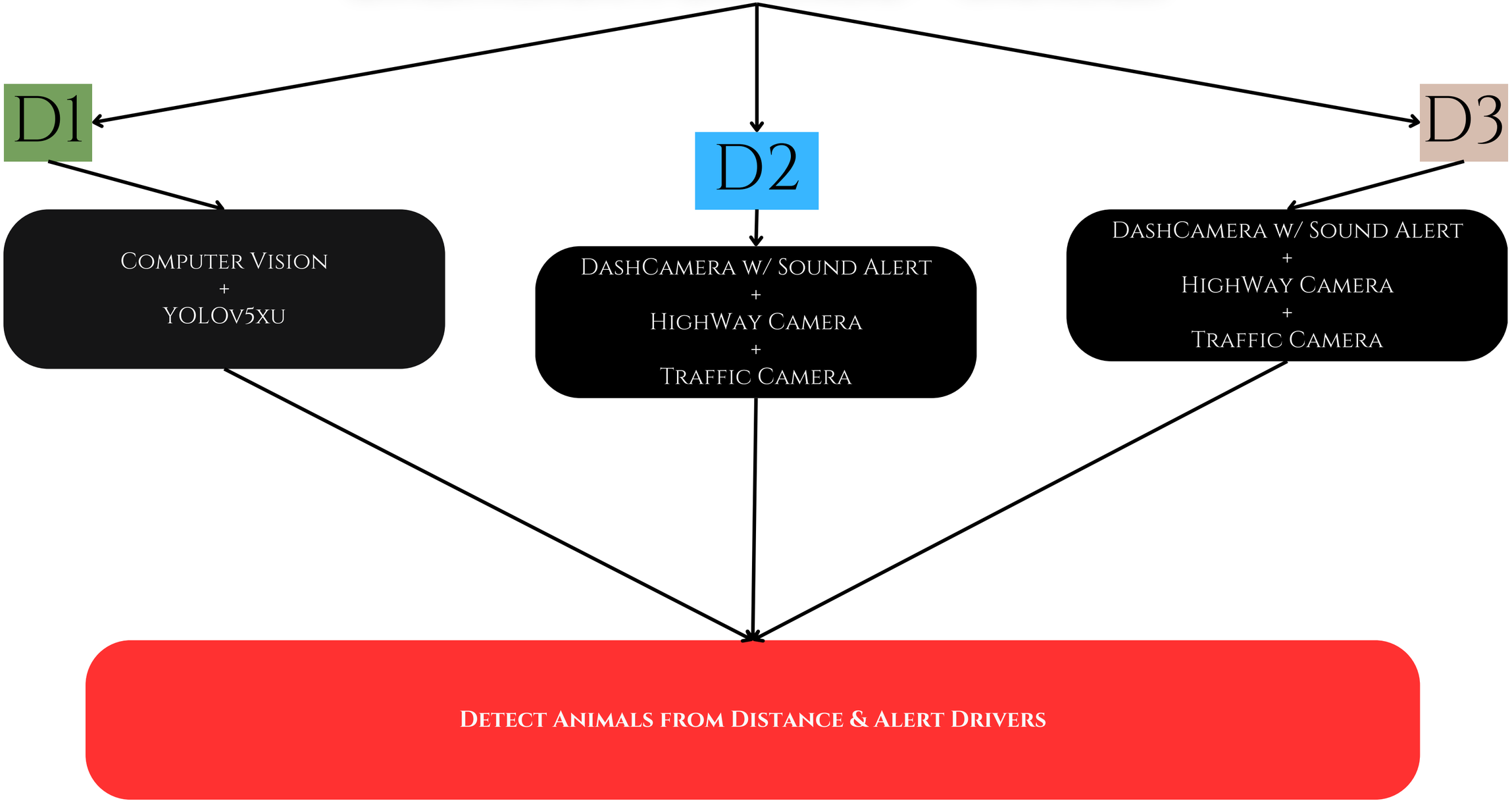
Multilayered Approach

Our idea has multilayered approach of 3 dimensions, aiming to improve the condition of stray and wild animals in an advanced way:

1. Dimension 1: **Computer Vision | YOLOv5xu**
2. Dimension 2: **CNN Model | DenseNet121** + **EfficientNet-B3**
3. Dimension 3: **IoT Integration** and **Vehicle Automation**



DashCam Footage + Images



Technical Overview

D1 | YOLOv5xu Model:

- Detects animals & objects in real-time.
- Identifies if the animal is dead or alive.
- Tracks animal **movement speed**.
- Detects **high-risk** sudden movements.



D2 | CNN Model:

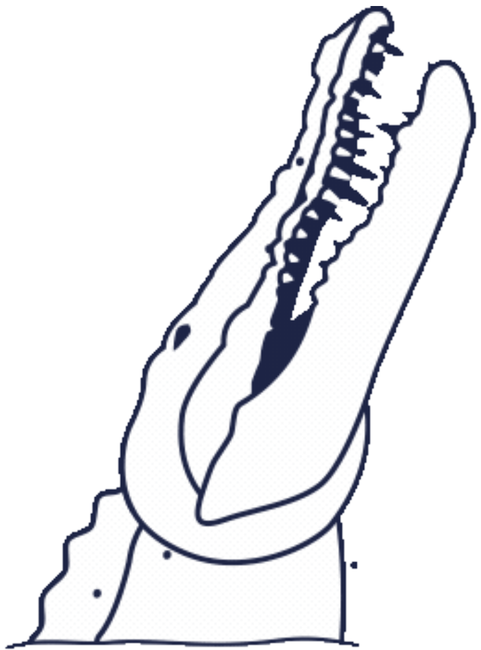
- Classifies animals as **dead** or **alive**, and alerts the driver on-the-go.
- 94%-98% accuracy w/ DenseNet & EfficientNet
- Integrated with Flask-based web interface.



Coding & Demonstration

+

Output



Dimension 1

Computer Vision

YOLOv5xu

1. Trained our model w/ YOLOv5xu within OpenCV
    a. Gave a stock footage of Stray Dogs from SSU for foundational training.

Output:





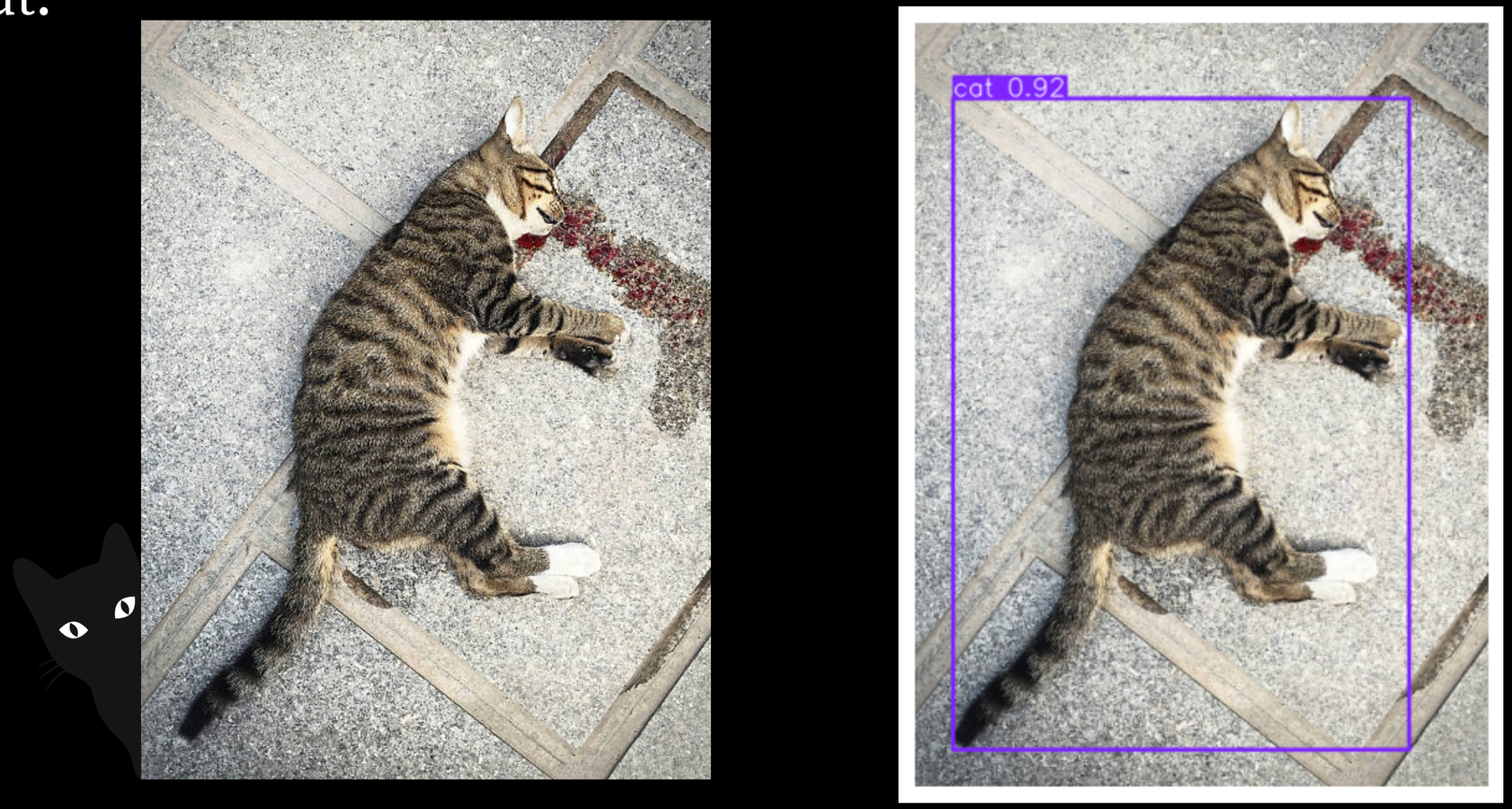
Dimension 1

Computer Vision

YOLOv5xu

2. Trained our model w/ YOLOv5xu for Object Identification within Image:
    a. Gave an image of a Flatlined Cat for foundational training.

Output:



Dimension 1

Computer

YoloV5

3. Trained our model w/ Farneback OpticalFlow by analyzing movement of animals to classify them as flatlined or alive, + their risk level if Alive:
    a. Gave a footage of a Flatlined Racoon on highway for foundational training.

Output:



Dimension 1

Computer Vision

Yolov5xu

4. Trained our model w/ Farneback OpticalFlow by analyzing the speed of animals to estimate their velocity if they run infront of the vehicle:
    a. Gave a footage of a vehicle DashCam w/ a Deer running at night for foundational training.

Output:



Dimension 1

Computer Vision

Yolov5xu

5. Trained our model w/ Alert System to notify the drivers, that triggers whenever an Animal is detected on the road with High Risk:
    a. Gave a footage of a vehicle DashCam w/ a puppy running in front a car out of nowhere, for foundational training.

Output:





Dimension 2

Computational Neural Network

DenseNet-121

- Used DenseNet121 and EfficientNetB3 (Functional API) as base model.

Data Augmentation:

- Resize to (**224, 224**)
- Random horizontal flip and rotation of **30 degrees**
- Width & height shift (0.2)
- Shear & zoom augmentation
- Normalization with **preprocess_input**

Model Architecture Modifications:

- GlobalAveragePooling2D()
- Dense(256, ReLU) → BatchNorm → Dropout(0.4)
- Dense(128, ReLU) → BatchNorm → Dropout(0.3)
- Dense(1, Sigmoid)
- L2 regularization used (kernel_regularizer=l2(0.01)

Training Process:

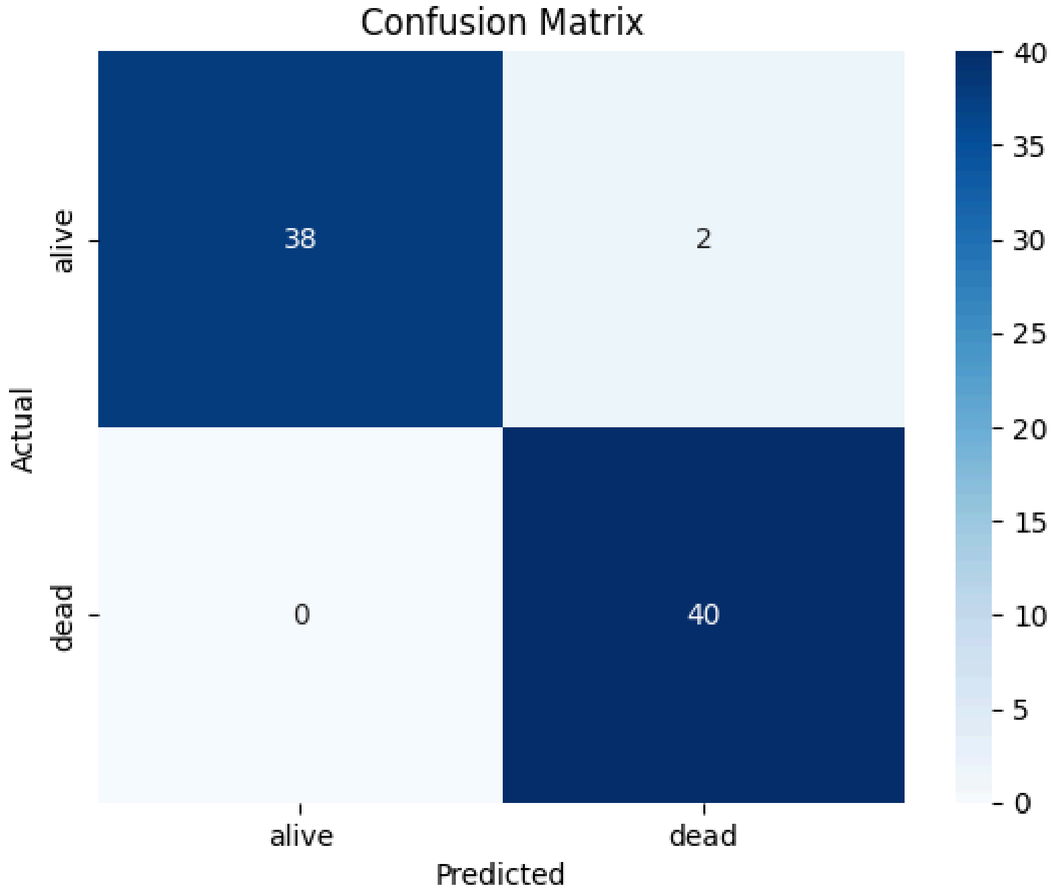
- **20 epochs** with **Adam Optimizer** (factor = 0.2)
- Early stopping mechanism (patience: **5**)

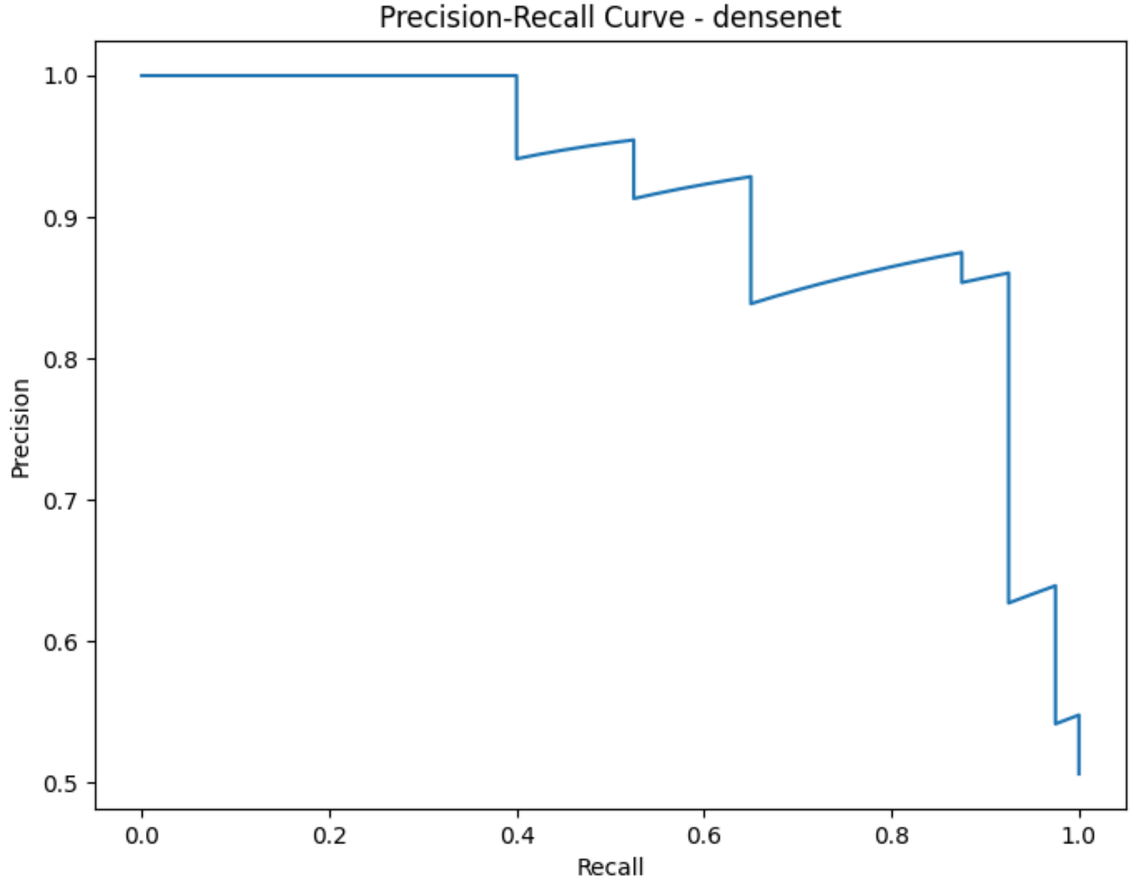


Dimension 2

Computational Neural Network

DenseNet-121

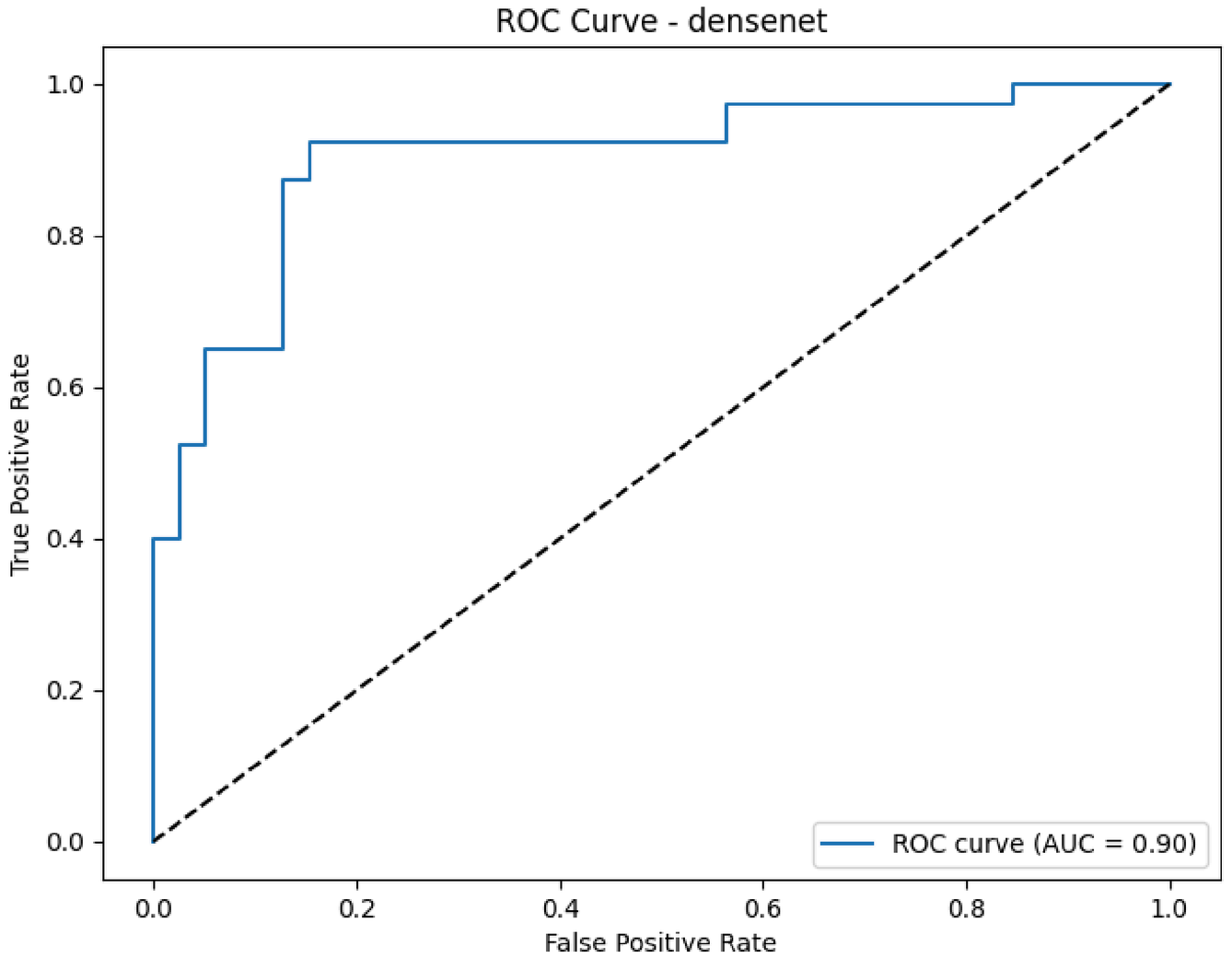




Dimension 2

Computational Neural Network

DenseNet-121

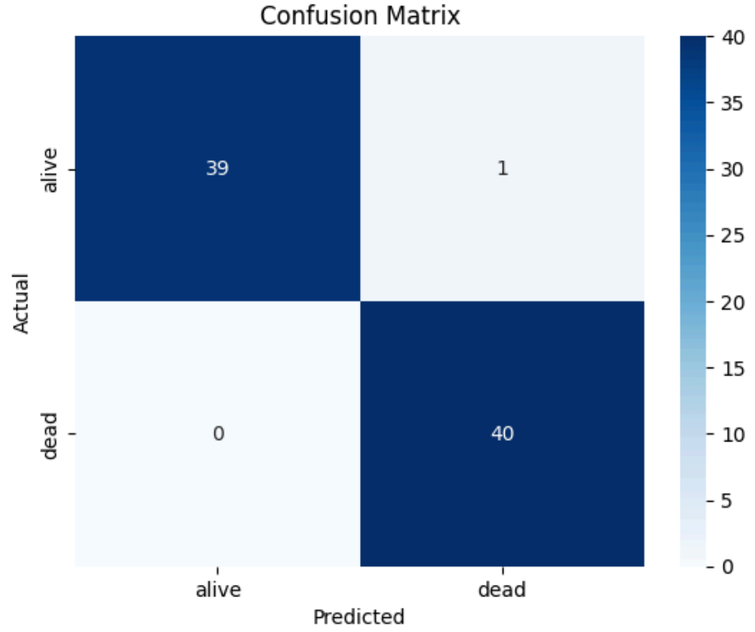


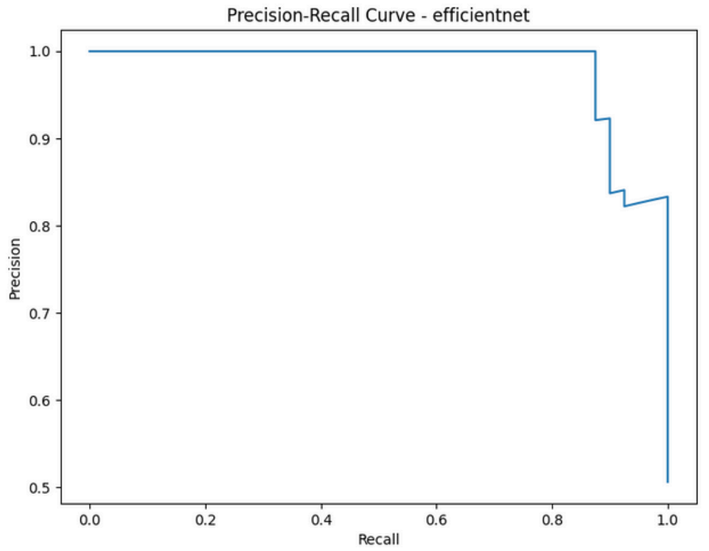
Dimension 2

Computational Neural Network

EffiecientNEt-B3

- Used Sequential-1 EfficientNet_B3 as base model.
  - Its a part of the **EfficientNet family**, known for its balance between efficiency and accuracy.
- The Data Augmentation, Model Architecture, & Training Process is **same** as DenseNet_121 as they were trained together.

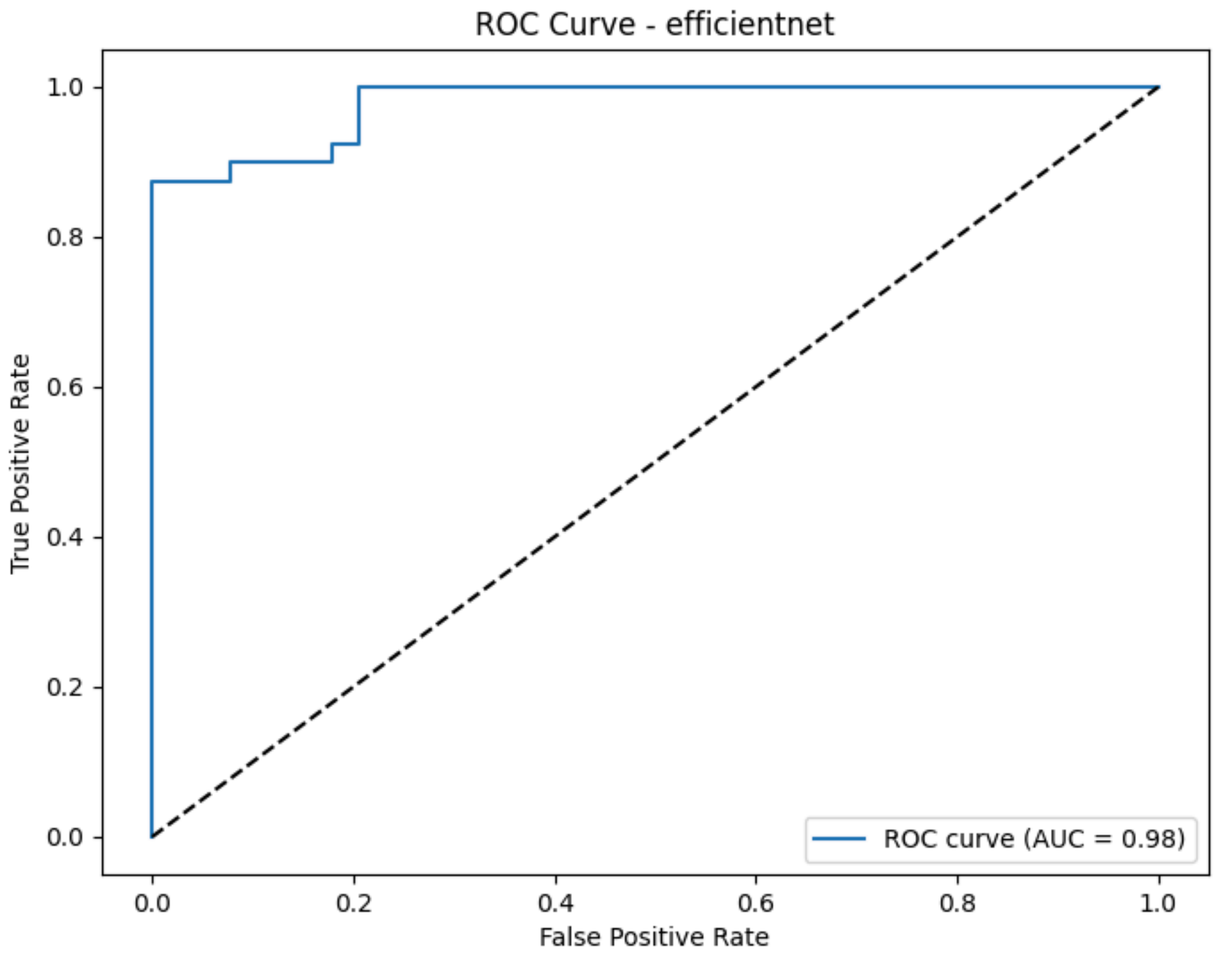




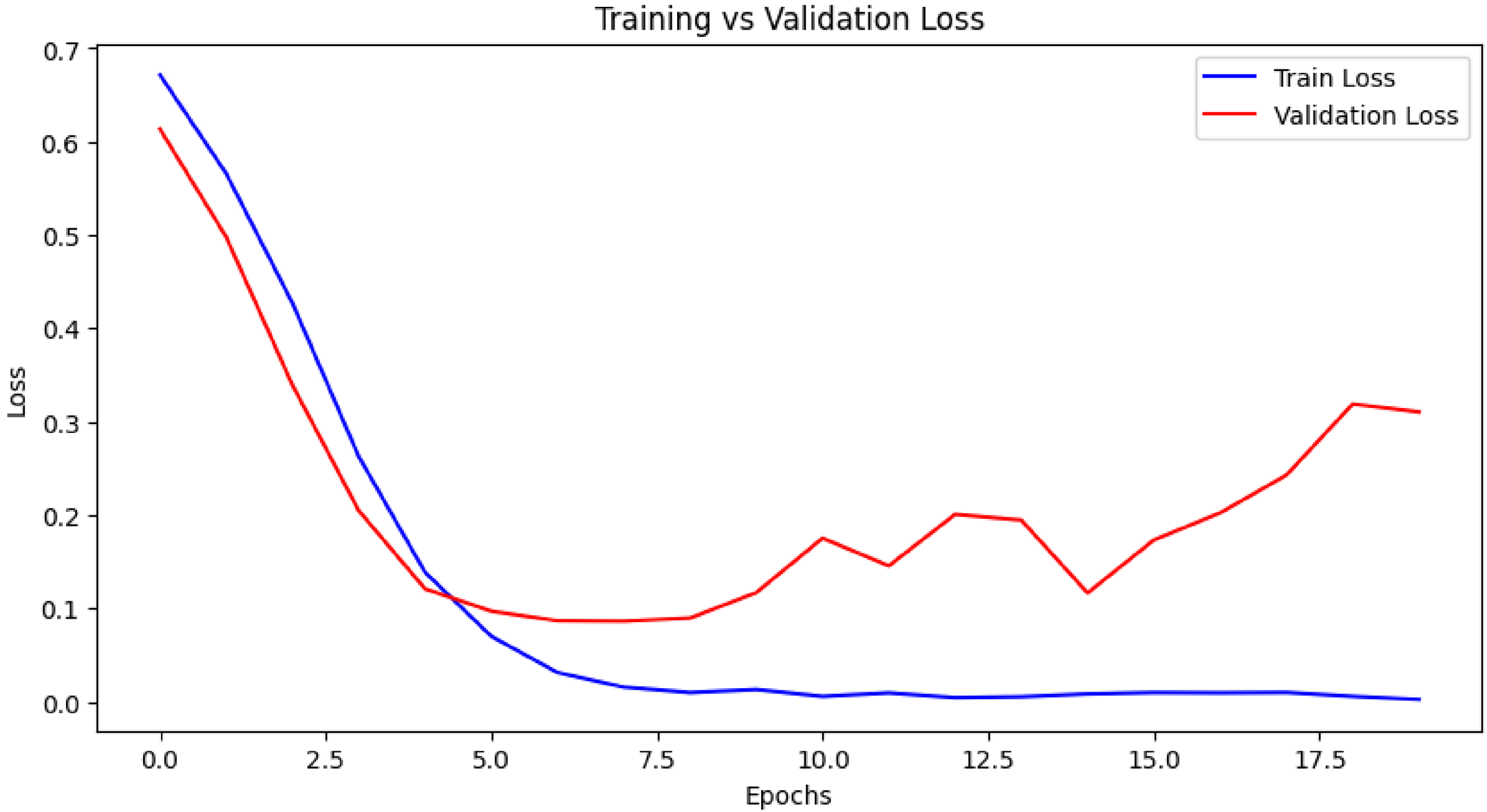
Dimension 2

Computational Neural Network

EffiecientNEt-B3



The Models initially showed an **overfitting problem** with the perfect **accuracy of 100%** each, then we augmented the data, regularized w/ L2 regularization & DropOut and also implemented EarlyStopping w/ patience of 5.



CNN Model Building + Execution

jupyter v3 PyTorch Last Checkpoint: 4 hours ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

```python
# Compute Performance Metrics
def compute_metrics(model, loader):
    model.eval()
    y_true, y_pred_probs = [], []

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            outputs = model(images).squeeze().cpu().numpy()
            y_true.extend(labels.numpy())
            y_pred_probs.extend(outputs)

            y_pred = [1 if p > 0.5 else 0 for p in y_pred_probs]

    metrics = {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred),
        "recall": recall_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred),
        "auc": roc_auc_score(y_true, y_pred_probs)
    }

    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix')
    plt.show()

    return metrics

# Train and Evaluate both models
for model_type in ["densenet", "efficientnet"]:
    print(f"Training {model_type}...")
    model = get_model(model_type)
    history = train_model(model, train_loader, val_loader, model_type)
    test_metrics = compute_metrics(model, test_loader)
    pd.DataFrame([test_metrics]).to_csv(f"model_results_{model_type}.csv", index=False)

print("Training & Evaluation Completed! 🚀")
```

[ ]:

CNN Model Building + Execution

jupyter v3 PyTorch Last Checkpoint: 4 hours ago

Trusted

File Edit View Run Kernel Settings Help

Code JupyterLab Python 3 (ipykernel)

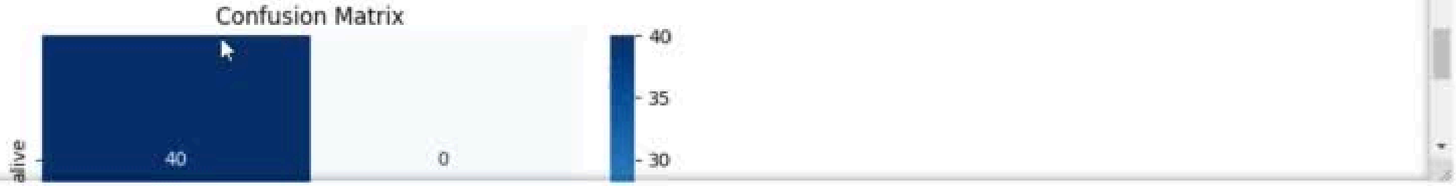
```
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix')
    plt.show()

    return metrics

# Train and Evaluate both models
for model_type in ["densenet", "efficientnet"]:
    print(f"Training {model_type}...")
    model = get_model(model_type)
    history = train_model(model, train_loader, val_loader, model_type)
    test_metrics = compute_metrics(model, test_loader)
    pd.DataFrame([test_metrics]).to_csv(f"model_results_{model_type}.csv", index=False)

print("Training & Evaluation Completed! 🚀")
```

```
Current LR: 0.000100
Epoch 13/20 - Train Loss: 0.0287, Val Loss: 0.0707, Val Acc: 0.9792
Current LR: 0.000100
Epoch 14/20 - Train Loss: 0.0237, Val Loss: 0.0694, Val Acc: 0.9792
Current LR: 0.000100
Epoch 15/20 - Train Loss: 0.0134, Val Loss: 0.1051, Val Acc: 0.9583
Current LR: 0.000050
Epoch 16/20 - Train Loss: 0.0109, Val Loss: 0.1684, Val Acc: 0.9792
Current LR: 0.000050
Early stopping triggered.
```



[ ] 1

CNN Model Building + Execution

kaggle.com/code/lightbkaggel/rerun-rhpa/edit

RHPA | YouTube | Maps | loktrax | lokesh0patra | Google Keep | Chat GPT | Lokesh Patra | academics | work | entertainment | FUN | ai | Check | Monkeytype | All Bookmarks

reRun RHPA Draft saved

File Edit View Run Settings Add-ons Help

Share | Save Version 0

Turn off internet
Accelerator
Environment Preferences

ode

Draft Session (28m)

None
GPU T4 ×2
GPU P100
TPU VM v3-8

```
import os
import shutil
import numpy as np
from sklearn.model_selection import tra
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.models import load_model
from tensorflow.keras.layers import Dense, Flatten, Dropout, BatchNormalization, GlobalAveragePooling2D
from tensorflow.keras.regularizers import l2
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc, confusion_matrix, classification_report
import seaborn as sns
import cv2
```

```
# Define Image Size & Paths
img_size = (224, 224)
dataset_dir = "/kaggle/input/roadkill-hotspot-and-policy-advocacy/RHP"
train_dir = "/kaggle/working/train"
test_dir = "/kaggle/working/test"
```

```
# Ensure train and test directories exist
os.makedirs(train_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)
for category in ["dead", "alive"]:
    os.makedirs(os.path.join(train_dir, category), exist_ok=True)
    os.makedirs(os.path.join(test_dir, category), exist_ok=True)
```

Notebook

Input

+ Add Input | Upload

DATASETS

- roadkill-hotspot-and-policy-advocacy
  - RHP
    - alive
    - dead

Output (85.3MiB / 19.5GiB)

- /kaggle/working
  - dog_status_model.h5
  - test
    - ... 1 more
  - train
    - ... 1 more

Table of contents



Model Comparison

Model	Accuracy	Precision	Recall	F1 Score	AUC Score
DenseNet-121	0.975	0.952380952	1	0.975609756	1
EfficientNet-B3	0.9875	0.975609756	1	0.987654	0.995625
Better Model	97.5	95.23809524	100	97.56097561	100
	98.75	97.5609756	100	98.7654	99.5625

Interfaces



Species Detection

jupyter Species Detection w YOLOv5xu Last Checkpoint: 6 hours ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python 3 (ipykernel)

Interrupt Kernel I I
Restart Kernel... 0, 0
Restart Kernel and Clear Outputs of All Cells...
Restart Kernel and Run up to Selected Cell...
Restart Kernel and Run All Cells...
Restart Kernel and Debug...
Reconnect to Kernel
Shut Down Kernel
Shut Down All Kernels...
Change Kernel...

[1]:
```
import cv2
import nump
import torc
import grad
import os
from ultral
```

[2]:
```
# Load YOLO
device = "c
print(f"Usi
model = YOL                                    xu\yolov5xu.pt").to(device)
```
```
Using device: cuda
```

[3]:
```
# Output directory for processed videos
output_dir = r"H:\Jupyter\Gradio Environment"
```

[4]:
```
def get_unique_filename():
    """Generates a unique filename for the processed video."""
    i = 1
    while True:
        new_filename = os.path.join(output_dir, f"Grad_Processed_AnmlDet_Video_{i}.mp4")
        if not os.path.exists(new_filename):
            return new_filename
        i += 1
```

[5]:
```
def process_video(input_video):
    """Processes the input video using YOLOv5xu and saves it with a unique filename."""

    cap = cv2.VideoCapture(input_video)

    # Get video properties
    frame_width = int(cap.get(3))
    frame_height = int(cap.get(4))
    fps = int(cap.get(cv2.CAP_PROP_FPS))

    # Generate a unique output filename
    output_path = get_unique_filename()

    # Define codec and create VideoWriter object
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))
```

Animal Image Detection

RHPA | YouTube | Maps | loktrax | lokesh0patra | Google Keep | Chat GPT | Lokesh Patra | academics | work | entertainment | FUN | ai | Check | Monkeytype | All Bookmarks

jupyter Image Animal Detection Last Checkpoint: 6 hours ago

File Edit View Run Kernel Settings Help

Kernel Unknown | Trusted

Code | JupyterLab | Python 3 (ipykernel)

```
from ultralytics import YOLO
```

```
# Load YOLOv5xu model on GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
model = YOLO(r"H:\Jupyter\IBM ICE Day 3\RHPA Comp. Vis. YOLOv5xu\yolov5xu.pt").to(device)
```

```
def detect_animals(image_path):
    """Processes an uploaded image using YOLOv5xu and returns the result."""

    # Read the image
    image = cv2.imread(image_path)

    # Run YOLOv5xu model on the image
    results = model(image)

    # Process results and draw bounding boxes
    for result in results:
        im_array = result.plot()
        im_rgb = cv2.cvtColor(im_array, cv2.COLOR_BGR2RGB)

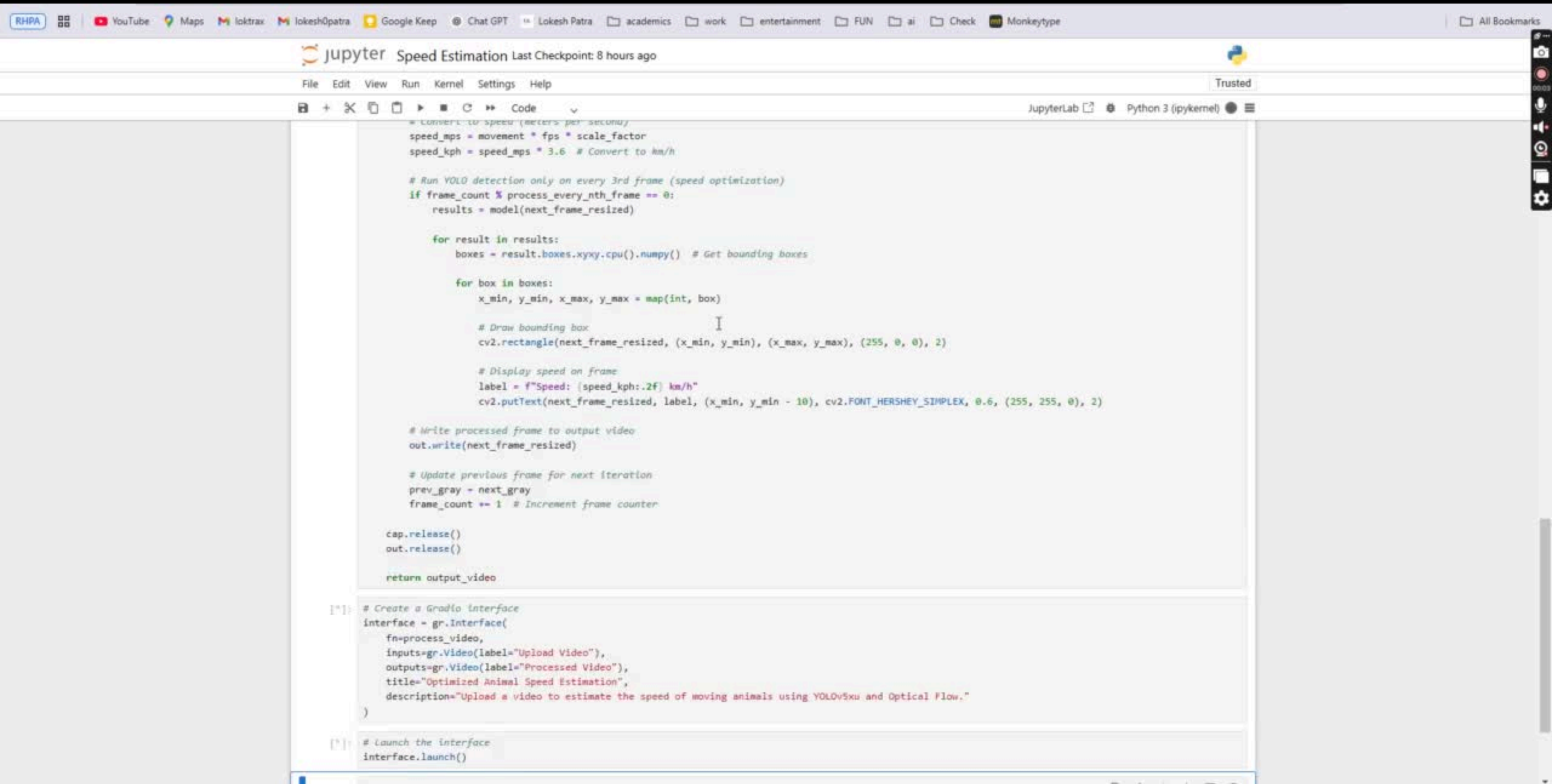
    return im_rgb  # Return processed image
```

```
# Gradio Interface
interface = gr.Interface(
    fn=detect_animals,
    inputs=gr.Image(type="filepath", label="Upload Image"),
    outputs=gr.Image(label="Processed Image"),
    title="Animal Detection using YOLOv5xu",
    description="Upload an image to detect animals using YOLOv5xu.",
)

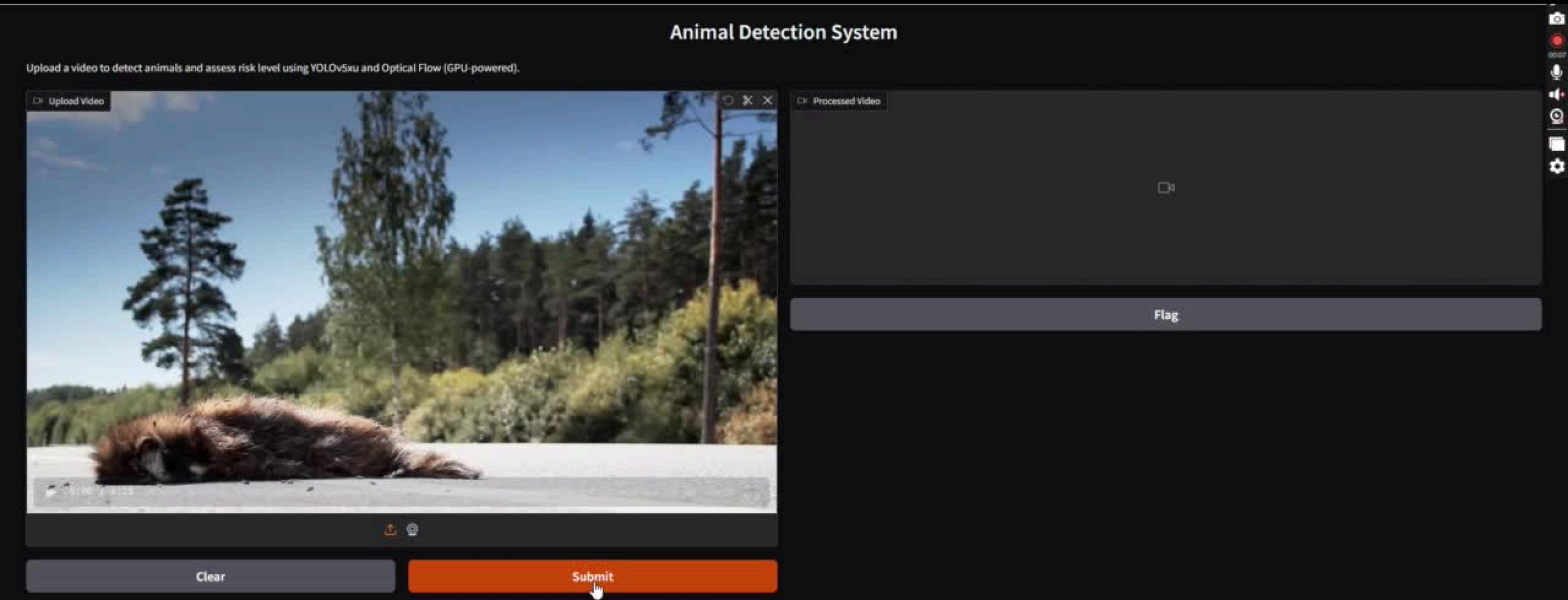
# Launch the interface
interface.launch()
```

[ ]:

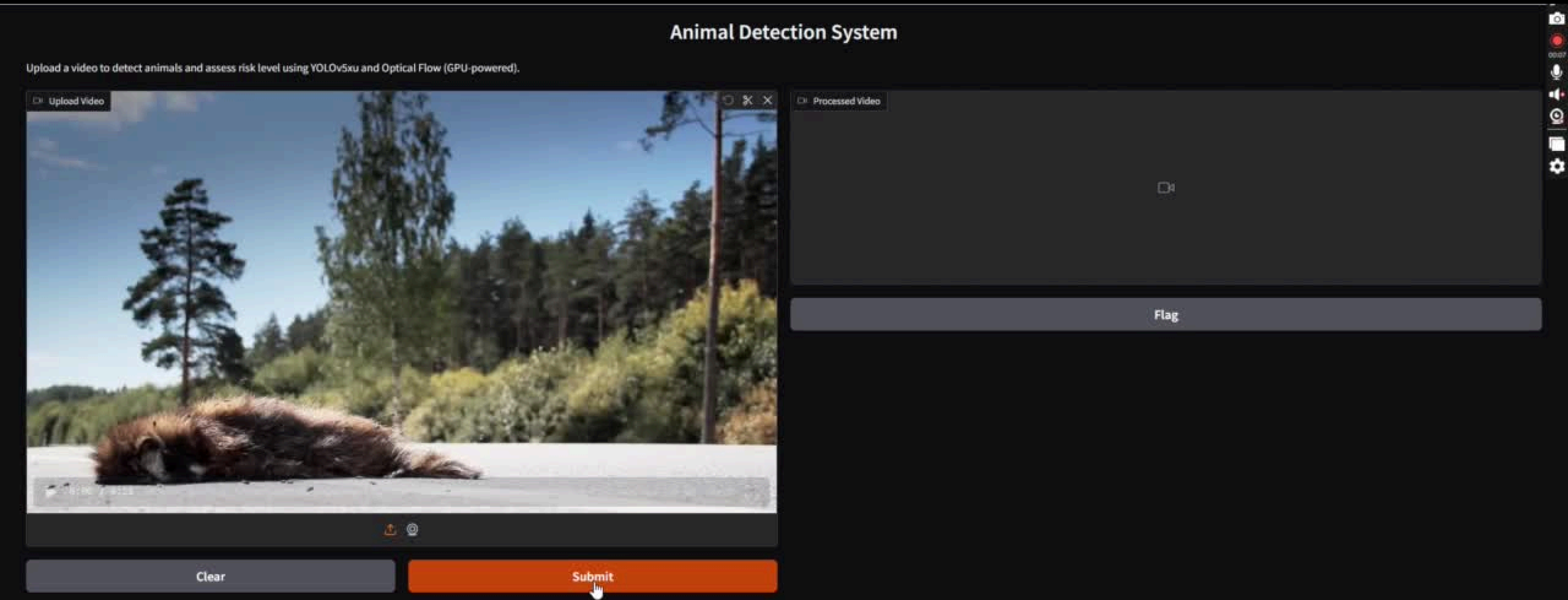
Speed Estimation



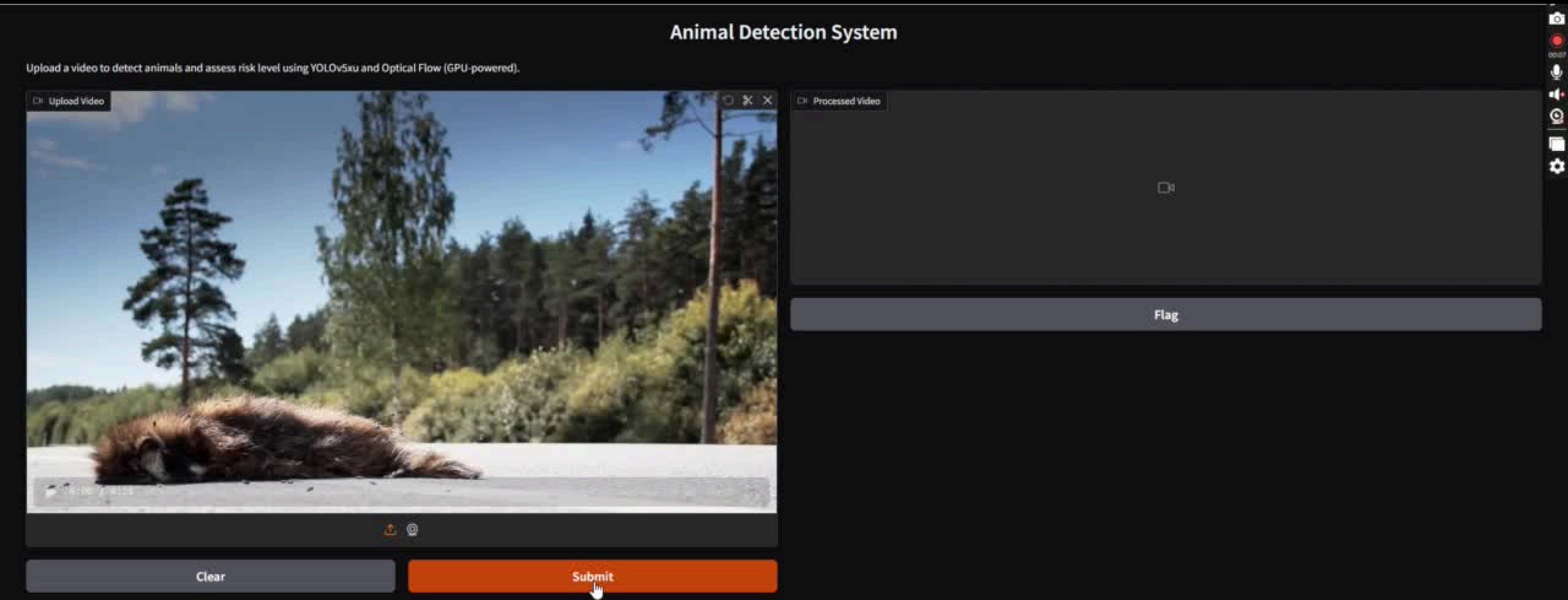
Flatlined /ALive Animal Detetction w/ OpticalFlow



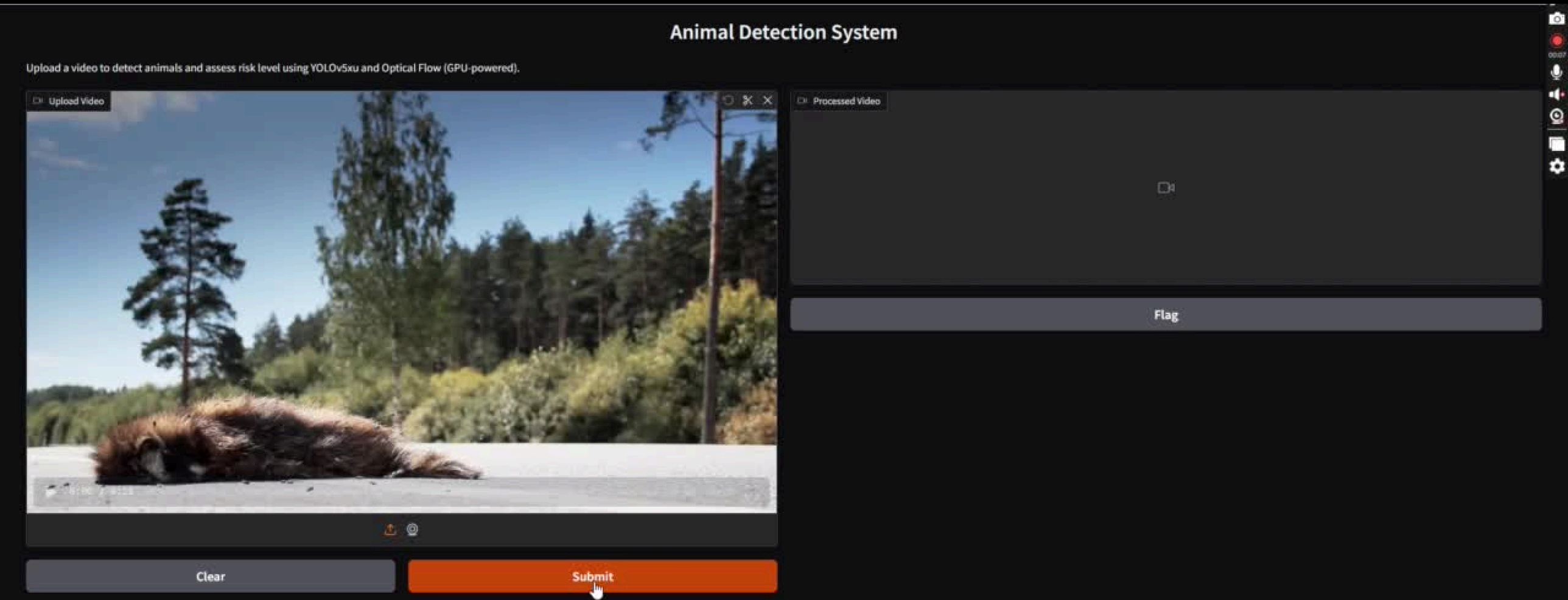
Flatlined /ALive Animal Detetction w/ OpticalFlow



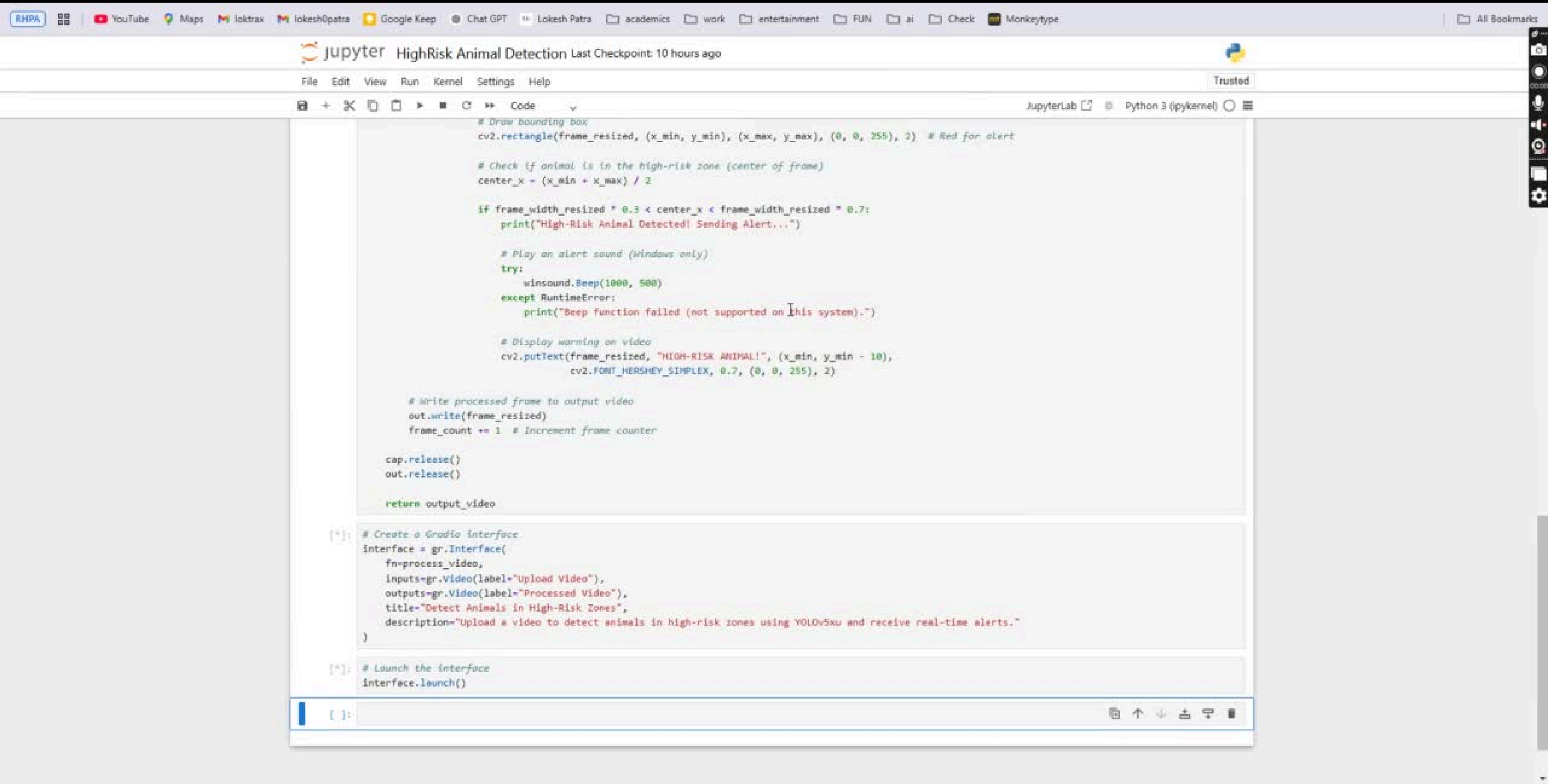
Flatlined /ALive Animal Detetction w/ OpticalFlow



Flatlined /ALive Animal Detetction w/ OpticalFlow



High Risk Animal Detection w/ Alert System



Advanced Features

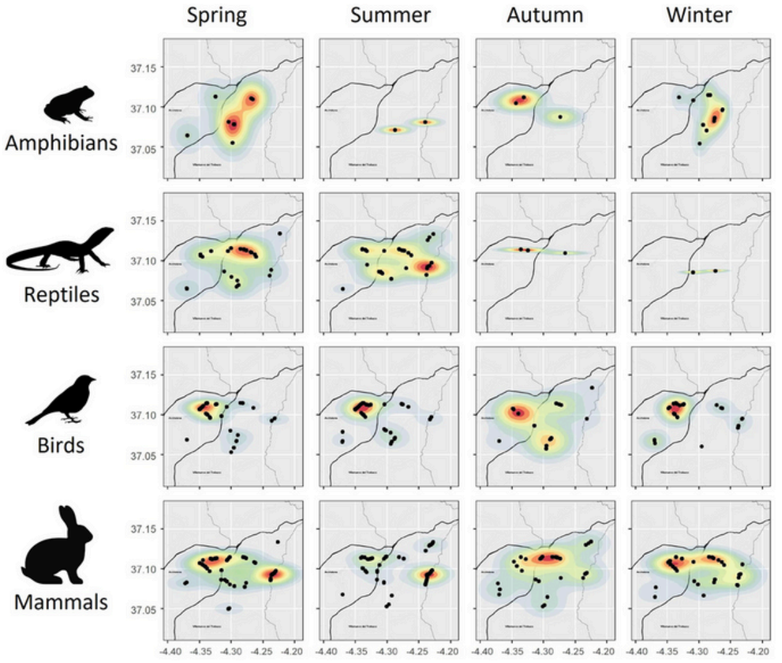
- **Heatmaps** & Predictive Analytics.
- **Edge AI** & **TinyML** Integration for real-time processing.
- **IoT-based Smart Signage** for real-time driver alerts.



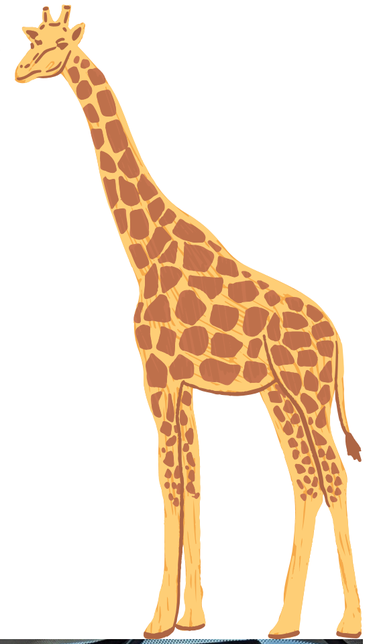
How It Works

- **AI-powered** dashcams & highway cameras **detect animals in real time.**
- High-risk animal detection **triggers** driver alerts.
- **Smart signage** warns drivers in **high-risk zones.**
- **Heatmaps** identify roadkill-prone areas.





Implementation Strategy



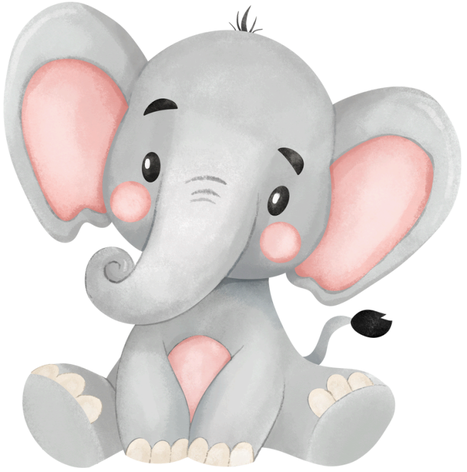
- **Government Partnerships**: Integrate with traffic surveillance.
- **Automobile Industry**: AI-powered dashcams.
- **Public Awareness**: Digital campaigns & road signage.



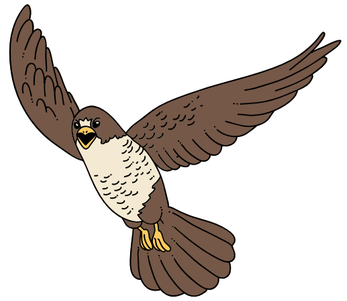


Business & Sustainability Plan

- **Royalty-Based Model**: License to governments & auto manufacturers.
- **Freemium Model**: Free for authorities, premium analytics for enterprises.
- **Long-term Goal**: Establish AI-driven road safety globally.

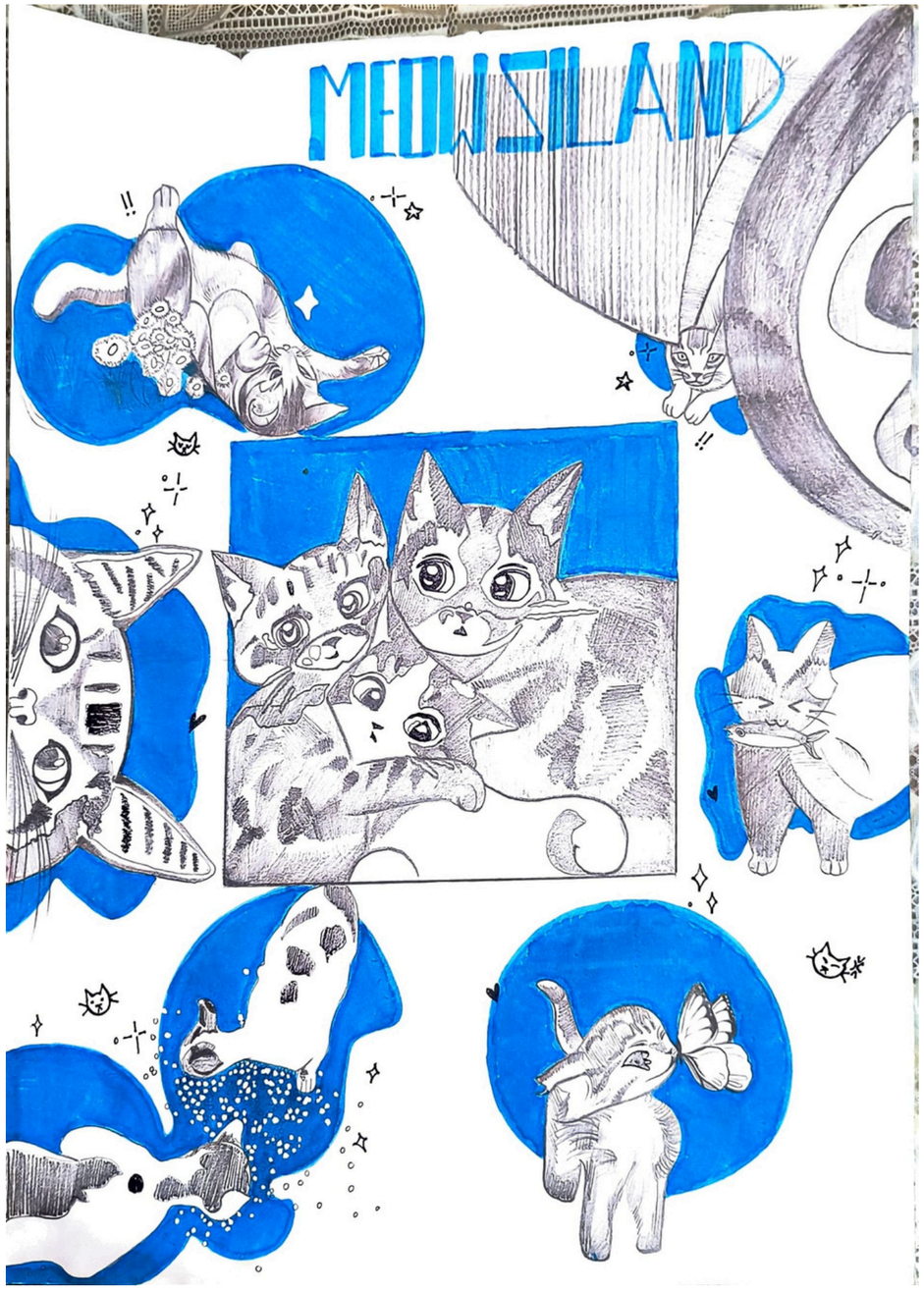


Expected Impact

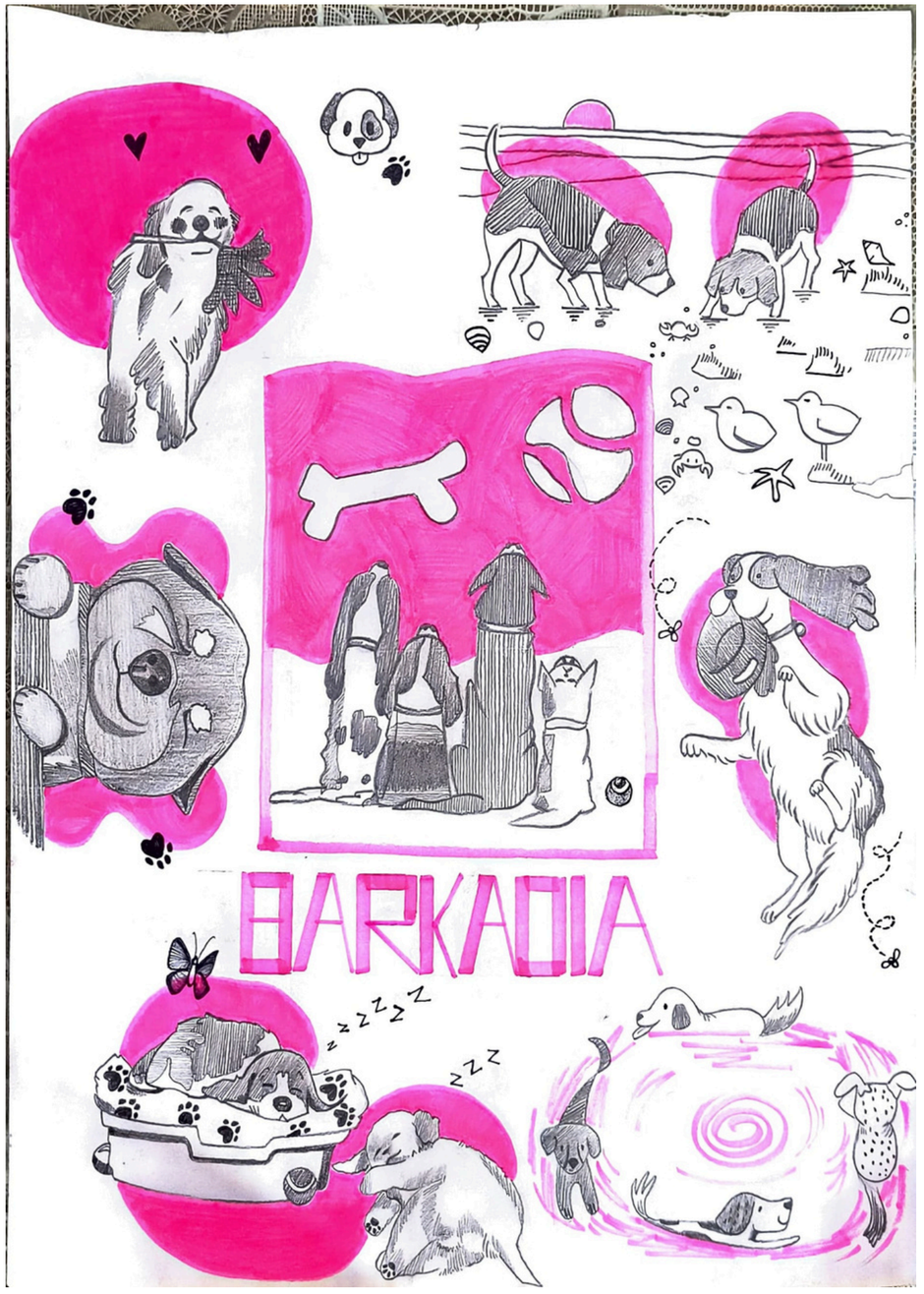


- Reduce roadkill incidents by **60-80%.**
- Improve **safety for animals** & **humans.**
- Advance **AI-based transportation** solutions.
- Drive **policy advocacy for animal safety**.





MEHEAK



DRIVE SAFE, THEY TOO HAVE A FAMILY!



